Personal AI Assistant — Full Codebase Documentation

A Reverse Engineer's Guide to Every File, Function, and Data Flow

Table of Contents

1. Project Overview

This is a **multi-agent AI personal assistant** built in Python. It has nine specialist chat agents (each an expert in one domain) plus two autonomous CrewAI pipelines for research and data analysis.

A supervisor router reads your message and automatically sends it to the right chat agent. No commands needed — just talk naturally. For heavy research jobs, the "Crew Mode" in the web UI dispatches a multi-agent pipeline that runs in the background.

```
"add buy groceries to my tasks"        → goes to Alfred    (task agent)
"what's the tech news today?"          → goes to Najwa     (news agent)
"explain Python decorators"            → goes to Linus     (coding agent)
"add expense 50000 for lunch"          → goes to Mansa     (budget agent)
"research CRISPR gene editing"         → goes to Ferry     (research agent)
```

Two ways to run it: - **CLI mode** → chat agents in your terminal (`python main.py`) - **Web mode** → browser UI at `http://localhost:8000` (`python server.py`) + Crew Mode for pipelines

---

1.1 Recent Updates (Changelog)

2026-06-17 — History & latest-transaction fixes

Mansa (Finance) — "Transaksi Terakhir" now shows the true latest entry. - `recent_transactions` in `GET /api/finance/dashboard` and `GET /api/budget/summary` is now sorted by `created_at` (date **+ time**), with a `date + " 00:00"` fallback for legacy rows, instead of by `date` alone. - *Why:* sorting by day-granularity `date` plus Python's stable sort pushed a just-added transaction to the bottom of its same-day group, so the newest entry did not appear first. Each transaction already carries `created_at = "YYYY-MM-DD HH:MM"` (set in `add_income` / `add_expense`). - The dashboard already auto-refreshes every 5 s (`setInterval(loadDashboard, 5000)` in `static/finance/index.html`) and after each Mansa chat reply, so the "Transaksi Terakhir" list updates without a manual reload.

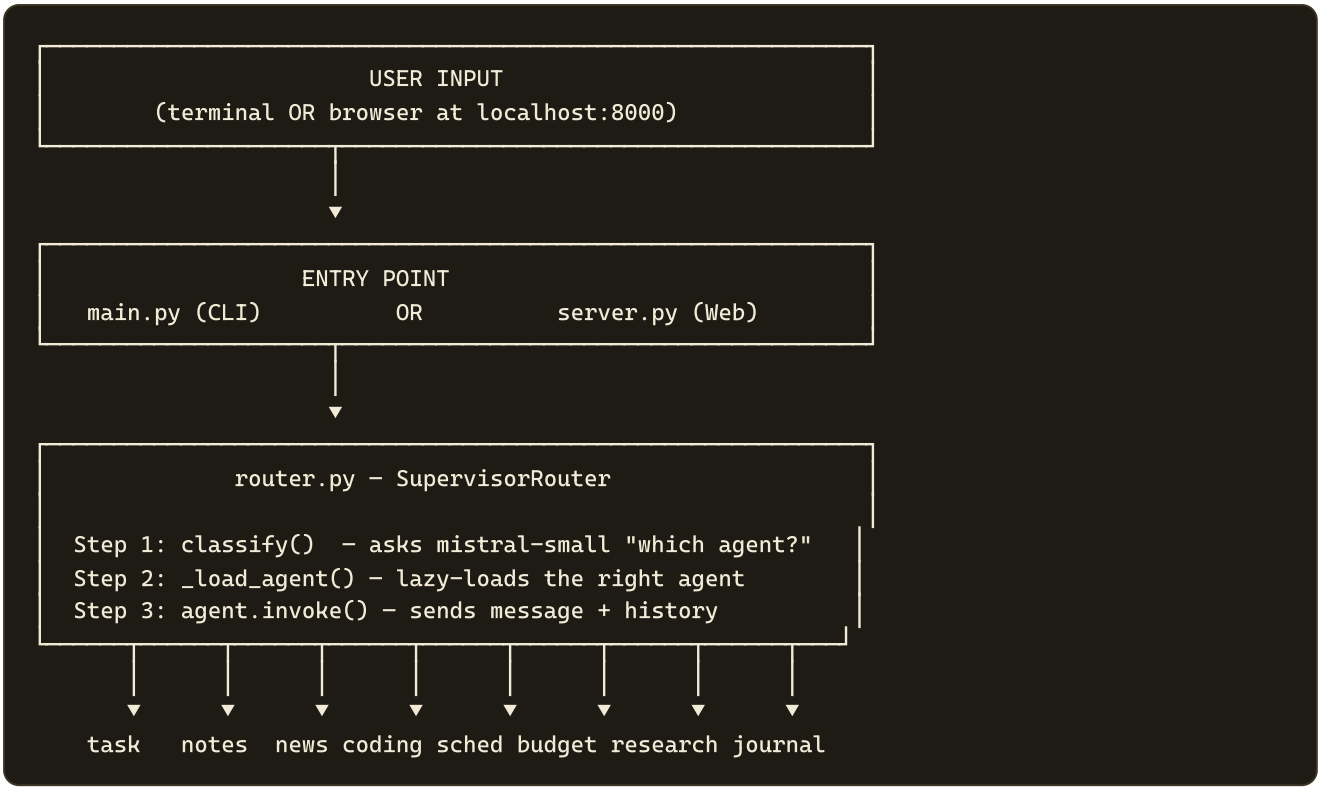
Lavoisier (Fitness) — previously-consumed food history is backfilled. - `_sync_food_logs_from_md()` in `server.py` now scans **both** the active vault folder (`<vault>/Lavoiser Agent`, i.e. `AI Data/My AI/Lavoiser Agent`) **and** the legacy `AI Data/Lavoiser Agent/` folder (helper: `_lavoiser_dirs()`). - It parses every `FoodSummary_YYYY-MM-DD.md` and merges any **missing** day into `data/food_log.json` (existing days are never overwritten). This backfilled the older Apr–early-May history that the old single-folder sync had skipped.

Dostoyevsky (Journal) — history is returned in both the dashboard and chat. - *Dashboard* (`GET /api/journal/dashboard`): Reflecta journals **conversationally** (`**Kamu:** / **Journal:**` dialogue). The dashboard used to discard every `**Kamu:**` section, leaving entries showing ~5 words. It now keeps the conversation as the entry body; speaker labels and `## HH:MM · JOURNAL` timestamp headers are stripped only for the preview/word-count. - *Chat agent* (`agents/dostyevsky_agent.py`): a new **Phase 0** instructs Reflecta to call `list_journal_entries` + `get_mood_history` once at session start (and `read_journal_entry` / `search_journal` on demand) so it recalls past entries. The tool list in the prompt was corrected to the real tool names. - Journal data lives in `<OBSIDIAN_VAULT_PATH>/Dostoyevsky Agent` (= `AI Data/My AI/Dostoyevsky Agent`), one `Journal_YYYY-MM-DD.md` per day plus `Emotion_YYYY-MM-DD.json` from the background emotion agent.

---

2. Architecture — The Big Picture

Chat Agent Flow



CrewAI Pipeline Flow (Crew Mode)

```
Browser Crew Mode → POST /api/crew/kickoff
    │
    ▼
server.py: background thread → crewai_agents.build_crew(topic)
    │
    ▼
IbnAlHaythamPipeline.kickoff()
    │
    ├─ Phase 1 sequential: Scout → Filter
    ├─ Phase 2 parallel:   IdeaGen ‖ Validator
    └─ Phase 3 sequential: Synthesizer → Critic → Writer
    │
    ▼
Output files written → GET /api/crew/status/{job_id} polls until done
```

Key design principle: every agent is independent. Each agent has its own: - System prompt (personality + instructions) - Tool list (what it can DO) - Chat history (last 20 messages, per agent)

3. Directory Structure

```
ai_python/
│
├── main.py              ← CLI entry point (chat agents only)
├── server.py            ← FastAPI web server (chat + crew + scraper endpoints)
├── router.py            ← Supervisor: classifies and routes chat messages
├── crewai_agents.py     ← CrewAI pipelines: Ibn Al-Haytham (7-agent) + DataAnalyst (3-agent)
│
├── agents/              ← LangChain/LangGraph chat agents
│   ├── base.py          ← Shared factory: build_agent(system_prompt, tools, temperature)
│   ├── task_agent.py    ← Alfred: to-do list manager
│   ├── notes_agent.py   ← Cicero: note-taking + wiki integration
│   ├── news_agent.py    ← Najwa: 24h news briefings
│   ├── coding_agent.py  ← Linus: programming tutor
│   ├── schedule_agent.py← CalCore: Google Calendar
│   ├── budget_agent.py  ← Mansa: personal finance (18 tools, multi-account + net worth)
│   ├── research_agent.py← Ferry: deep autonomous research
│   ├── davinci_agent.py ← Da Vinci: creative thinking
│   └── journal_agent.py ← Dostoyevsky: personal journaling
│
├── tools/               ← LangChain tool definitions
│   ├── task_tools.py    ← add/list/complete/delete/update tasks
│   ├── notes_tools.py   ← create/read/search/update/delete notes + URL fetch
│   ├── news_tools.py    ← DuckDuckGo search (last 24h)
│   ├── schedule_tools.py← Google Calendar API
│   ├── budget_tools.py  ← 18 tools: accounts, net worth, investments, budget goals, recurring
│   ├── research_tools.py← deep web search, iterative search, URL fetch, compile report
│   ├── wiki_tools.py    ← Obsidian-style wiki (write/query/ingest/update/lint)
│   └── autoresearch_tools.py ← persistent experiment log for research programs
│
├── social_scraper/      ← Multi-platform social media scraper (crew_type: "scraper")
│   ├── __init__.py
│   ├── agents/
│   │   ├── scrapegraph_harvester.py  ← ScrapeGraphAI v2 + Mistral harvester (active)
│   │   ├── crawl4ai_harvester.py     ← Legacy crawl4ai harvester (kept as reference)
│   │   └── summarizer_agent.py       ← Mistral AI per-platform summary generator
│   └── data/raw/                     ← Raw scraped JSON per platform/timestamp
│
├── data/                ← JSON flat-file storage
│   ├── tasks.json       ← list of task objects
│   ├── notes.json       ← list of note objects
│   └── budget.json      ← dict: {accounts, transactions, budget_goals, investments,
│                                 net_worth_history, recurring}
│
├── credentials/         ← Google OAuth (never commit)
│   ├── credentials.json ← Downloaded from Google Cloud Console
│   └── token.pickle     ← Auto-generated after first OAuth login
│
├── static/
│   ├── index.html       ← Redirect stub (points to /index/)
│   ├── index/           ← Multi-file React frontend (main app)
│   │   ├── index.html   ← HTML shell (loads JSX + CSS via Babel CDN)
│   │   ├── app.jsx      ← Root App component, theme, keyboard shortcuts
│   │   ├── views.jsx    ← ChatView, DashboardView, Sidebar, RightPanel, Masthead
│   │   ├── overlays.jsx ← CommandPalette, CrewDrawer, ResultModal
│   │   ├── data.jsx     ← AGENTS config, AGENT_CLUSTERS, MOCK data, API helpers
```

```
│   │   ├── icons.jsx    ← SVG icon components
│   │   └── styles.css   ← All styles (CSS variables, dark/light theme)
│   ├── finance/         ← Mansa Finance Dashboard (standalone page at /finance)
│   │   └── index.html   ← Paper & Ink design; light/dark mode; 6 sections + chat panel
│   ├── avatars/         ← Agent avatar images (one JPG per agent)
│   └── uploads/         ← CSV uploads for DataAnalyst pipeline
│
├── AI Data/             ← Agent output files, wiki, research reports
│   ├── Ferry Agent/     ← Ibn Al-Haytham pipeline output (task1_scout.txt … task6_final_report.
│   └── Social Scraper/  ← Per-platform AI summaries (platform_YYYY-MM-DD.md)
│
├── docs/superpowers/    ← Design specs and implementation plans
│   ├── specs/
│   └── plans/
│
├── .env                 ← API keys (never commit)
├── requirements.txt     ← Python dependencies
├── CLAUDE.md            ← Project instructions for Claude Code
├── cassanovaL_instruction.md  ← Manual guide: adding new agents
└── DOCUMENTATION.md     ← This file
```

---

4. Data Flow — How a Message Travels

Let's trace exactly what happens when you type **"add task: buy milk"** in the browser.

Step 1 — Browser sends HTTP POST

```
POST http://localhost:8000/api/chat
Body: { "message": "add task: buy milk", "agent": "task" }
```

The frontend always sends the currently-selected agent ID in the body.

Step 2 — server.py receives the request

```
@app.post("/api/chat")
async def chat(req: ChatRequest):
    supervisor = get_supervisor()
    agent_name, response = supervisor.chat_direct(req.agent, req.message)
    return {"agent": agent_name, "response": response}
```

Because `req.agent` is `"task"`, it calls `chat_direct()`, skipping auto-classification.

Step 3 — router.py loads and calls the agent

```
def chat_direct(self, agent_name, user_message):
    agent = self._load_agent("task")        # lazy-loads task agent on first call
    history = self._chat_histories["task"]  # [] initially, grows over session
    messages = history + [HumanMessage(content=user_message)]
    response = agent.invoke({"messages": messages})
    answer = response["messages"][-1].content
    # appends to history, trims to last 20
    return "task", answer
```

Step 4 — LangGraph agent runs the tool loop

The agent (a `CompiledStateGraph` from LangChain 1.x) does: 1. Sends messages to `mistral-large-latest` 2. The LLM sees the system prompt + chat history + "add task: buy milk" 3. The LLM decides to call `add_task(title="buy milk", priority="medium")` 4. LangGraph executes the tool → writes to `data/tasks.json` 5. Tool returns: `"Task added! ID:ab3f1c2d | \"buy milk\" | Priority:medium"` 6. LLM formulates final reply: `"Done! I've added 'buy milk' to your task list."`

Step 5 — Response travels back

```
agent.invoke() → router → server.py → HTTP 200 JSON → browser → React state → rendered in chat
```

---

5. Entry Points

main.py — CLI Mode

Purpose: Runs the assistant as a terminal chatbot.

```
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")
```

> **Why this line?** Windows uses `cp1252` encoding by default. This forces `stdout` to UTF-8 so emoji and non-ASCII characters don't crash the terminal.

```
router = SupervisorRouter()
```

> Creates the router. This connects to Mistral AI immediately (validates the API key).

```
while True:
    user_input = input("You: ").strip()
    agent_name, answer = router.chat(user_input)
    icon = AGENT_ICONS.get(agent_name, "[ AGENT   ]")
    print(f"\n{icon}\n{answer}\n")
```

> The main loop. `router.chat()` does **auto-classification** (unlike the web mode which sends the agent explicitly). The icon shows which agent answered.

Special commands recognized before routing: - `quit` / `exit` / `keluar` / `bye` → exits - `help` → prints example commands - Empty input → skipped

---

server.py — Web Mode

Purpose: HTTP server that serves the React frontend AND exposes the AI as a REST API.

Tech stack: FastAPI + Uvicorn

```
app = FastAPI(title="OmniSync API", version="1.0.0")
```

CORS Middleware

```
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)
```

> Allows requests from any origin. Required because the browser frontend and API are on the same origin normally, but this also allows testing from tools like Postman or a different port.

Lazy Supervisor

```
_supervisor = None

def get_supervisor():
    global _supervisor
    if _supervisor is None:
        from router import SupervisorRouter
        _supervisor = SupervisorRouter()
    return _supervisor
```

> The supervisor is only created when the first request comes in. This prevents startup delay — the server starts instantly, and the AI initializes on the first message.

Endpoints

Method	Path	What it does
`POST`	`/api/chat`	Send message to AI agent, get response
`GET`	`/api/tasks`	Get all tasks + stats (for sidebar panel)
`GET`	`/api/notes`	Get recent notes + total count
`GET`	`/api/budget/summary`	Get balance, monthly totals, recent transactions
`GET`	`/{any_path}`	Serve `static/index.html` (SPA fallback)

The SPA Catch-All Route

```
@app.get("/{full_path:path}")
async def serve_frontend(full_path: str):
    return FileResponse("static/index.html")
```

> Any URL that doesn't match an API route (e.g. `/`, `/settings`, `/anything`) returns `index.html`. This is the standard pattern for single-page applications.

---

6. The Brain — router.py

Purpose: Decides which agent handles each message. Acts as a traffic cop.

AGENT_REGISTRY

```
AGENT_REGISTRY = {
    "task":     "Managing to-do lists, tasks, reminders, and deadlines",
    "notes":    "Writing notes, saving information, summarizing articles or research URLs",
    "news":     "Latest news, current events, headlines, recent updates",
    "coding":   "Programming help, code explanation, debugging, tutorials, tech questions",
    "schedule": "Calendar, meetings, events, appointments, schedule management",
    "budget":   "Money, expenses, income, spending, finance, budget, cashflow",
}
```

> This dictionary does two things: 1. Lists which agent names are valid 2. Provides descriptions used in the classification prompt

SupervisorRouter Class

```
self.llm = ChatMistralAI(model="mistral-small-latest", temperature=0.0)
```

> Uses the **small** Mistral model for routing because: - Classification is a simple task (pick one of six words) - `temperature=0.0` means completely deterministic — no randomness - Small model = faster + cheaper (routing happens on every message)

```
self._agents: dict = {}
self._chat_histories: dict = {name: [] for name in AGENT_REGISTRY}
```

> - `_agents` starts empty — agents are created only when first needed (**lazy loading**)
> - `_chat_histories` stores a separate conversation history for each agent so context is preserved per-agent

classify() — The Classifier

```
def classify(self, message: str) → str:
    agent_list = "\n".join(f"- {name}: {desc}" for name, desc in AGENT_REGISTRY.items())
    prompt = CLASSIFY_PROMPT.format(agent_list=agent_list, message=message)
    response = self.llm.invoke([HumanMessage(content=prompt)])
    agent_name = response.content.strip().lower().split()[0]
    return agent_name if agent_name in AGENT_REGISTRY else "task"
```

> Sends a prompt to Mistral that looks like: `Available agents: - task: Managing to-do lists ... - notes: Writing notes ... ... User message: "what's bitcoin price today?" Agent:` The LLM replies with just one word: `news`. If the reply isn't a valid agent name, it defaults to `task`.

_load_agent() — Lazy Loading

```
def _load_agent(self, name: str):
    if name not in self._agents:
        from agents.task_agent import create_task_agent
        self._agents[name] = create_task_agent()
    return self._agents[name]
```

> Python imports are cached by the interpreter. This pattern means: - First call to `_load_agent("task")` → imports module + creates agent - Subsequent calls → returns the already-created agent from `self._agents` - Agents you never use are never loaded (saves memory + API calls)

chat() vs chat_direct()

	`chat()`	`chat_direct()`
Classification	Yes (calls `classify()`)	No (uses agent name directly)
Used by	`main.py` (CLI)	`server.py` (web)
Why	Terminal doesn't show which agent is selected	Browser always shows which agent tab is active

Chat History Management

```
history.append(HumanMessage(content=user_message))
history.append(AIMessage(content=answer))
if len(history) > 20:
    self._chat_histories[agent_name] = history[-20:]
```

> - History is stored as LangChain message objects (`HumanMessage` , `AIMessage`)
> - Capped at 20 messages to prevent infinite context growth (would slow API calls + cost more tokens)
> - Each agent has **isolated** history — switching from task to notes doesn't mix conversations

---

7. Agent Factory — agents/base.py

Purpose: One function that builds any agent. All six agents use this.

```
from langchain.agents import create_agent

def build_agent(system_prompt: str, tools: list, temperature: float = 0.2):
    llm = ChatMistralAI(
        model="mistral-large-latest",
        temperature=temperature,
        mistral_api_key=api_key,
    )
    return create_agent(llm, tools, system_prompt=system_prompt)
```

Why `mistral-large-latest` here but `mistral-small-latest` in the router? - The router just classifies (pick one word from six) → small model is fine - The agents must understand nuanced requests, write code, manage data, format outputs → large model needed

What does `create_agent()` return? It returns a `CompiledStateGraph` — a LangGraph state machine that: 1. Starts with the message list 2. Calls the LLM 3. If LLM wants to call a tool → executes it 4. Feeds tool result back to LLM 5. Repeats until LLM gives a final answer (no more tool calls) 6. Returns final state with all messages

The `temperature` parameter: - `0.0` = fully deterministic (router) - `0.1` = nearly deterministic (news — factual reporting) - `0.2` = slight creativity (default for task, notes, schedule, budget) - `0.3` = more varied (coding — allows multiple explanation styles)

---

8. The Chat Agents

Each agent file has two things: a **system prompt** and a **factory function**.

task_agent.py

Agent name: Alfred

```
def create_task_agent():
    return build_agent(SYSTEM_PROMPT, TASK_TOOLS)
```

- **Tools available:** `add_task`, `list_tasks`, `complete_task`, `delete_task`, `update_task`
- **Data store:** `data/tasks.json`
- **Bilingual:** responds in Indonesian if the user writes in Indonesian

notes_agent.py

Agent name: Cicero - **Tools available:** `create_note`, `list_notes`, `read_note`, `search_notes`, `update_note`, `delete_note`, `fetch_and_summarize_url`, plus wiki tools (`write_research_to_wiki`, `query_wiki`, `ingest_source`, `update_wiki_entity`, `lint_wiki`) - **Special ability:** fetches URLs and summarizes content; writes findings to an Obsidian-style wiki - **Data store:** `data/notes.json` + `AI Data/wiki/`

news_agent.py

Agent name: Najwa - **Tools available:** `get_recent_news`, `get_top_headlines` - **Data store:** none — uses live DuckDuckGo search - **Low temperature (0.1):** news reporting should be factual, not creative - **Time filter:** DuckDuckGo configured with `time="d"` (last 24 hours only)

coding_agent.py

Agent name: Linus - **Unique:** defines its own `search_documentation` tool inline, not in `tools/` — tightly coupled to coding-specific site filters - **Temperature 0.3:** coding explanations benefit from some variation

schedule_agent.py

Agent name: CalCore - **Tools available:** `list_upcoming_events`, `get_today_schedule`, `create_event`, `delete_event`, `update_event` - **Timezone:** Asia/Jakarta (UTC+7) by default - **Requires:** `credentials/credentials.json` from Google Cloud Console - **Auth flow:** On first use, opens a browser for OAuth. Saves token to `credentials/token.pickle`.

budget_agent.py

Agent name: Mansa - **Tools available (18 total):** - *Transactions (6):* `add_income`, `add_expense`, `get_balance`, `list_transactions`, `get_monthly_summary`, `delete_transaction` - *Accounts (3):* `add_account`, `list_accounts`, `update_account_balance` - *Net Worth (2):* `get_net_worth`, `snapshot_net_worth` - *Budget Goals (2):* `set_budget_goal`, `check_budget_goals` - *Investments (3):* `add_investment`, `update_investment_price`, `get_portfolio_summary` - *Recurring (2):* `add_recurring`, `get_recurring` - **Data store:** `data/budget.json` — dict schema with 6 keys (see Section 10) - **Finance Dashboard:** Mansa's sidebar entry opens `/finance` (standalone page) instead of a chat tab; the chat panel on `/finance` still routes to the `budget` agent - **Localized:** uses Rupiah (Rp) by default, responds in Indonesian if addressed in Indonesian - **max_tokens:** 2048 (bumped from 1024 to handle rich multi-account responses)

research_agent.py

Agent name: Ferry A 4-phase autonomous deep-research agent. Runs layered searches, cross-references sources, and produces a structured report. - **Tools available:** deep web search, iterative search, URL fetch, multi-source synthesis, compile report + full wiki integration suite - **Phases:** (1) Scope & plan, (2) Layered search execution, (3) Quality checks, (4) Final report - **Wiki integration:** after every research session, automatically writes results to `AI Data/wiki/` and updates related entity pages - **AutoResearch:** reads/logs/updates a persistent experiment program that tracks which research strategies produce the most accurate plans - **Output:** saved to `AI Data/Ferry Agent/` as dated markdown reports - **Temperature 0.1:** highly factual, near-deterministic

davinci_agent.py + journal_agent.py

Da Vinci — creative thinking, brainstorming, lateral thinking exercises. Pure LLM reasoning, no external tools.

Dostoyevsky (Reflecta) — personal journaling. Tools: `write_journal_entry`, `read_journal_entry`, `list_journal_entries`, `search_journal`, `get_mood_history` (+ habit/obsidian/autoresearch tools). Data store: `<OBSIDIAN_VAULT_PATH>/Dostoyevsky Agent/` (= `AI Data/My AI/Dostoyevsky Agent/`), one `Journal_YYYY-MM-DD.md` per day. Loads recent history at session start (Phase 0) so it can reference past entries. See the Changelog.

---

9. Tools Layer

Tools are Python functions decorated with `@tool` from LangChain. The decorator: 1. Reads the function's docstring → becomes the tool description the LLM sees 2. Reads the type hints → tells the LLM what arguments to provide 3. Wraps the function → makes it callable by the LangGraph agent loop

task_tools.py

Pattern used by all JSON-based tools:

```
TASKS_FILE = "data/tasks.json"

def _load() → list:
    with open(TASKS_FILE, "r", encoding="utf-8") as f:
        return json.load(f)

def _save(data: list):
    with open(TASKS_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)
```

> `_load()` and `_save()` are private helpers (underscore prefix = convention for "internal use"). Every read operation loads the whole file; every write rewrites the whole file. Simple but works fine at personal-assistant scale.

Task data structure:

```
{
  "id": "ab3f1c2d",
  "title": "buy milk",
  "priority": "medium",
  "due_date": "2025-04-10",
  "status": "pending",
  "created_at": "2025-04-04 10:30"
}
```

Tool	What it does
`add_task(title, priority, due_date)`	Appends new task object with UUID ID
`list_tasks(filter_status, filter_priority)`	Filters and formats task list
`complete_task(task_id)`	Sets `status = "completed"`
`delete_task(task_id)`	Removes task by ID
`update_task(task_id, title, priority, due_date)`	Updates any field by ID

ID generation:

```
"id": str(uuid.uuid4())[:8]
```

> Generates a random UUID and takes the first 8 characters (e.g. `"ab3f1c2d"`). Short enough for the LLM to type in tool calls, unique enough for personal-scale data.

---

notes_tools.py

Note data structure:

```
{
  "id": "7e2a9b1f",
  "title": "Python async/await notes",
  "content": "async functions are coroutines ... ",
  "tags": ["python", "async", "programming"],
  "created_at": "2025-04-04 09:00",
  "updated_at": "2025-04-04 09:00"
}
```

Tool	What it does
`create_note(title, content, tags)`	Creates note; tags is comma-separated string → split to list
`list_notes(tag_filter)`	Shows all notes or filtered by tag; truncates content at 80 chars
`read_note(note_id)`	Returns full content of one note
`search_notes(query)`	Case-insensitive search in title AND content
`update_note(note_id, ...)`	Updates any field + refreshes `updated_at`
`delete_note(note_id)`	Removes note
`fetch_and_summarize_url(url)`	Fetches page, strips HTML tags with regex, returns first 3000 chars

How URL fetching works:

```
import re
text = re.sub(r"<[^>]+>", " ", response.text)  # strip <html tags>
text = re.sub(r"\s+", " ", text).strip()         # collapse whitespace
return text[:3000]                                # LLM reads this and summarizes
```

> This is "dumb" HTML stripping — it removes tags but keeps all text including nav, footer etc. It works well enough for the LLM to extract the important content.

---

news_tools.py

```
_search = DuckDuckGoSearchAPIWrapper(time="d", max_results=8)
```

> `time="d"` = results from last day only. `max_results=8` = up to 8 results per query.

Tool	What it does
`get_recent_news(topic)`	Searches `"{topic} news today"`
`get_top_headlines()`	Runs 3 searches: tech, world, business news today

---

schedule_tools.py

Authentication flow:

```
def _get_calendar_service():
    creds = None
    if os.path.exists(TOKEN_FILE):          # 1. Try loading saved token
        creds = pickle.load( ... )
    if not creds or not creds.valid:
        if creds.expired and creds.refresh_token:
            creds.refresh(Request())         # 2. Auto-refresh if expired
        else:
            flow = InstalledAppFlow( ... )
            creds = flow.run_local_server()  # 3. Full OAuth (opens browser)
    pickle.dump(creds, open(TOKEN_FILE, "wb"))  # 4. Save for next time
    return build("calendar", "v3", credentials=creds)
```

> **Pickle** is Python's built-in serialization format. The OAuth credentials object (which isn't plain JSON) is saved/loaded as binary data.

Tool	What it does
`list_upcoming_events(days=7)`	Lists next N days of events
`get_today_schedule()`	Today's events only
`create_event(title, start, end, description)`	Creates event in primary calendar
`delete_event(event_id)`	Finds event by ID prefix, then deletes
`update_event(event_id, ... )`	Patch update on found event

Timezone: All events use `Asia/Jakarta` (UTC+7) by default.

Partial ID matching:

```
for e in events:
    if e["id"].startswith(event_id):  # match partial ID
```

> Google Calendar event IDs are very long strings. The UI shows only the first 12 characters. Tools search for events whose full ID starts with the partial ID provided.

budget_tools.py

`budget_tools.py` was fully rewritten from 6 to **18 tools** to support multi-account wealth management (inspired by Maybe Finance / Sure).

Schema migration: `_load()` detects the old flat list format (`[{type, amount, ...}]`) and auto-migrates to the new dict structure on first load — zero data loss, zero manual intervention.

Type constants:

```
LIABILITY_TYPES = {"credit_card", "loan"}
ASSET_TYPES     = {"checking", "savings", "e_wallet", "investment_account", "property", "other"}
```

Transaction data structure (in `transactions` list):

```
{
  "id": "3c7d8e9a",
  "type": "income | expense",
  "amount": 50000.0,
  "category": "food",
  "description": "lunch at warung",
  "date": "2025-04-04",
  "account": "BCA Tabungan",
  "created_at": "2025-04-04 12:30"
}
```

Account data structure (in `accounts` list):

```
{
  "name": "BCA Tabungan",
  "account_type": "checking | savings | e_wallet | credit_card | investment_account | loan | pro
  "balance": 5000000.0,
  "currency": "IDR",
  "created_at": "2026-04-01 09:00"
}
```

Net worth formula: `assets_total + investment_market_value − liabilities_total` where `investment_market_value = sum(qty × current_price)` per holding.

Account balance auto-update: When `account` param is provided to `add_income` / `add_expense`, the matching account balance is updated in-place. Expense on a liability account *increases* balance (debt grows); expense on an asset account *decreases* balance.

All 18 tools:

Group	Tool	What it does
Transactions	`add_income(amount, category, description, date, account)`	Records income; updates account balance
	`add_expense(amount, category, description, date, account)`	Records expense; updates account balance
	`get_balance()`	`sum(income) - sum(expense)` across all time
	`list_transactions(month, tx_type, account)`	Filter by month, type, and/or account
	`get_monthly_summary(month)`	Grouped totals by category for a month
	`delete_transaction(transaction_id)`	Remove a transaction by ID
Accounts	`add_account(name, account_type, balance, currency)`	Add a bank/wallet/credit account
	`list_accounts()`	All accounts grouped by type; assets vs liabilities totals
	`update_account_balance(account_name, new_balance)`	Sync balance manually from bank app
Net Worth	`get_net_worth()`	Assets − Liabilities + investment market value
	`snapshot_net_worth()`	Save net worth snapshot to `net_worth_history`
Budget Goals	`set_budget_goal(category, monthly_limit, month)`	Set spending cap per category
	`check_budget_goals(month)`	Actual vs goal per category with progress %
Investments	`add_investment(ticker, name, inv_type, quantity, buy_price, currency)`	Add stock/crypto/bond/reksadana
	`update_investment_price(ticker, current_price)`	Update market price for a holding
	`get_portfolio_summary()`	Holdings table: qty × price, P&L, total value
Recurring	`add_recurring(description, amount, category, frequency, next_date, account)`	Add recurring bill/subscription
	`get_recurring()`	Upcoming recurring sorted by next_date

Number formatting:

```
f"+{amount:,.0f}"   # → "+50,000" (comma thousands separator, no decimals)
```

Obsidian mirror (`_mirror`): After every write operation, re-generates the monthly markdown file in `AI Data/Mansa Agent/` including account balances and current net worth alongside the transaction log.

---

10. Data Storage

All data is stored as **JSON flat files** in `data/`. There is no database.

Why JSON files?

- Zero setup (no database server to install)
- Human-readable and editable
- Sufficient for personal-assistant scale (hundreds or low thousands of records)
- Easy to backup, sync, or inspect

File schemas

data/tasks.json — list of task objects

```
[
  {
    "id": "string (8 chars)",
    "title": "string",
    "priority": "high | medium | low",
    "due_date": "YYYY-MM-DD | empty string",
    "status": "pending | completed",
    "created_at": "YYYY-MM-DD HH:MM"
  }
]
```

data/notes.json — list of note objects

```
[
  {
    "id": "string (8 chars)",
    "title": "string",
    "content": "string (full text)",
    "tags": ["string", "..."],
    "created_at": "YYYY-MM-DD HH:MM",
    "updated_at": "YYYY-MM-DD HH:MM"
  }
]
```

data/budget.json — dict with 6 keys (auto-migrated from old flat-list format on first load)

```
{
  "accounts": [
    { "name": "string", "account_type": "checking|savings|e_wallet|credit_card|investment_accour
      "balance": 0.0, "currency": "IDR", "created_at": "YYYY-MM-DD HH:MM" }
  ],
  "transactions": [
    { "id": "string (8 chars)", "type": "income|expense", "amount": 50000.0,
      "category": "string", "description": "string", "date": "YYYY-MM-DD",
      "account": "string", "created_at": "YYYY-MM-DD HH:MM" }
  ],
  "budget_goals": [
    { "category": "string", "monthly_limit": 1500000.0, "month": "YYYY-MM", "created_at": " ... "
  ],
  "investments": [
    { "ticker": "BBCA", "name": "string", "inv_type": "stock|crypto|bond|reksadana|other",
      "quantity": 100.0, "buy_price": 8500.0, "current_price": 9200.0, "currency": "IDR" }
  ],
  "net_worth_history": [
    { "date": "YYYY-MM-DD", "net_worth": 0.0, "assets": 0.0, "liabilities": 0.0, "investments":
  ],
  "recurring": [
    { "id": "string", "description": "Spotify", "amount": 55000.0, "category": "Subscription",
      "frequency": "monthly|weekly|yearly", "next_date": "YYYY-MM-DD", "account": "string" }
  ]
}
```

---

11. CrewAI Pipelines — crewai_agents.py

Ibn Al-Haytham Research Pipeline (7 agents)

A phase-based hybrid research system. `build_crew(topic)` returns an `IbnAlHaythamPipeline` instance; call `.kickoff()` to run.

```
Phase 1 — Sequential
  Scout  (mistral-small)  → task1_scout.txt       topic map + [MODE: ACADEMIC/GENERAL/HYBRID]
  Filter (mistral-small)  → task2_filter.txt      curated sources (10–15 best)

Phase 2 — Parallel (ThreadPoolExecutor, max_workers=2)
  IdeaGen   (gemma-4)     → task3a_ideas.txt      hypotheses + novel angles
  Validator (gemma-4)     → task3b_validation.txt cross-checked claims, [⚠] flags

Phase 3 — Sequential
  Synthesizer (mistral-large) → task4_synthesis.txt  unified narrative
  Critic      (mistral-large) → task5_critique.txt   logic review
  Writer      (mistral-large) → task6_final_report.md final article with [Ref N] citations
```

Key implementation details: - `_read_phase_output(fname)` reads a phase output file, returns `""` on any OS error - Phase 2 context: filter output is embedded as a string in each task's `description` (sliced to 4000 chars) — cannot use CrewAI `context=[task]` across separate Crew objects - Error handling: if one

Phase 2 thread fails, Synthesizer proceeds with `[PARTIAL: ...]` note; if both fail, raises `RuntimeError` - Phase 2 timeout: 300 seconds per thread - LLM fallbacks: if `GEMMA4_API_KEY` absent, Phase 2 agents fall back to Mistral automatically

Auto-detect mode: Scout outputs one of three tags on its first line: - `[MODE: ACADEMIC]` → prioritise arXiv, PubMed, IEEE, Nature - `[MODE: GENERAL]` → prioritise news, industry reports, DuckDuckGo - `[MODE: HYBRID]` → balanced; also the default if Scout fails to classify

Search provider priority: LinkUp (deep) > Serper > DuckDuckGo (free)

DataAnalyst Crew (3 agents)

Cleans, analyzes, and visualizes uploaded CSV files. Triggered via `build_data_analyst_crew(filename, goal)`.

```
DataBot-Clean (mistral-small) → task1_data_clean.txt    cleaned dataset summary
DataBot-Stats (gemma-4)       → task2_stats_analysis.txt statistical analysis
DataBot-Viz   (gemma-4 2B)    → task2_report.md          chart descriptions
```

Upload CSVs via `POST /api/upload` → select in Crew Mode → launch.

Social Scraper Pipeline (`crew_type: "scraper"`)

A two-stage AI pipeline that harvests trending content from up to 12 social platforms and generates structured, per-platform summaries in Indonesian.

Stage 1 — ScrapeGraphHarvester (`social_scraper/agents/scrapegraph_harvester.py`)

Uses **ScrapeGraphAI v2** with Mistral as the LLM backend. For each platform:

```
SearchGraph(prompt)
    └── SearchInternetNode  – DuckDuckGo search for "{platform} trending today"
    └── GraphIteratorNode   – SmartScraperGraph on top 5 result pages (Mistral extraction)
    └── MergeAnswersNode    – Mistral merges all extracted data into one structured list
```

- Prompt is both the DuckDuckGo search query AND the LLM extraction instruction
- Output saved to `social_scraper/data/raw/<platform>/scrapegraph_<platform>_<ts>.json`
- 12 supported platforms: `youtube`, `tiktok`, `facebook`, `instagram`, `twitter`, `reddit`, `linkedin`, `medium`, `twitch`, `pinterest`, `quora`, `soundcloud`
- Optional `keywords` filter to focus on specific topics

Stage 2 — SummarizerAgent (`social_scraper/agents/summarizer_agent.py`)

Reads raw JSON output from Stage 1 and calls `mistral-large-latest` to generate a structured markdown summary per platform:

```
## Trending Topics | ## Konten Populer | ## Tema Utama | ## Insight
```

Text is cleaned (noise removal: UI labels, nav links, short lines) and truncated to 6000 chars before sending to Mistral.

Output: Per-platform summary tabs appear in ResultModal. Summaries are also persisted to `AI Data/Social Scraper/<Platform>_YYYY-MM-DD.md`.

Config reference (in `ScrapeGraphHarvester`):

```
{
    "llm": { "model": "mistralai/mistral-small", "temperature": 0.1 },
    "max_results": 5,   # top search result pages to fetch per platform
    "headless": True,   # Playwright browser for JS-heavy pages
}
```

> **First-time setup:** Run `playwright install chromium` after `pip install -r requirements.txt`.

---

12. Frontend — static/index/

The frontend is a **multi-file React app** loaded via Babel CDN (no build step). All files live in `static/index/`.

File responsibilities:

File	Responsibility
`index.html`	HTML shell — loads React, Babel, all JSX files, and `styles.css`
`app.jsx`	Root `App` component: state, theme, keyboard shortcuts (`Ctrl+K` = palette, `Esc` = close)
`views.jsx`	`Sidebar`, `Masthead`, `ChatView`, `DashboardView`, `RightPanel`
`overlays.jsx`	`CommandPalette`, `CrewDrawer` (Crew Mode), `ResultModal`
`data.jsx`	`AGENTS` config, `AGENT_CLUSTERS`, `CLUSTER_ORDER`, mock data, API helpers
`icons.jsx`	SVG icon components (`IcoX`, `IcoCheck`, `IcoRocket`, etc.)
`styles.css`	CSS variables (`--ink`, `--paper`, `--clay`, `--hue-*`), dark/light theme, all component styles

Agent Cluster System (`AGENT_CLUSTERS` in data.jsx):

Agents are grouped into 4 clusters displayed as filter tabs in the sidebar:

Cluster	Agents	Accent
`all`	All agents	`--c-fg`
`personal`	Alfred, Miyamoto, Mansa, Lavoisier, Dostoyevsky, Da Vinci	`--hue-alfred`
`research`	Najwa	`--hue-najwa`
`academic`	Cicero, Linus	`--hue-cicero`
`trading`	(Stock pipeline via Crew Mode)	`#e6a817`

Each agent in `AGENTS` has a `cluster` field that maps it to one of the cluster keys.

External Agent Routing:

Agents with a `url` field in their `AGENTS` config open that URL in a new tab instead of a chat tab. Currently Mansa (`budget`) has `url: '/finance'` , making its sidebar entry a link to the Finance Dashboard. The roster row shows `↗` (external arrow) instead of the status dot.

```
// data.jsx
budget: {
  name: 'Mansa', sub: 'Finance Dashboard', url: '/finance',
  ...
}

// views.jsx — renders differently for external agents
const isExternal = !!ag.url;
onClick={() => isExternal ? window.open(ag.url, '_blank') : setActive(k)}
```

State (all in `App`):

```
agKey        // active agent key ("task" | "notes" | "research" | ...)
tab          // "chat" | "dashboard"
msgs         // { [agKey]: Message[] } — separate history per agent
loading      // true while awaiting API response
showCmd      // CommandPalette open
showCrew     // CrewDrawer open
panelOpen    // RightPanel open (auto-opens at ≥1100px)
dash         // { tStats, tasks, budget, notes, notesTotal, recentTx }
```

CrewDrawer flow: 1. User picks pipeline type (Research / Data Analyst / Social Scraper) + enters topic 2. `POST /api/crew/kickoff` → receives `job_id` 3. `setInterval` polls `GET /api/crew/status/{job_id}` every 2.5s 4. On `status: "done"` → outputs displayed in `ResultModal` as file tabs

Crew Mode node display (research pipeline): - 7 nodes with phase separators (Phase 1 / Phase 2 with parallel badge / Phase 3) - Each node shows: number, agent name, role, LLM badge (`mistral-small` / `gemma-4` / `mistral-large`)

12.1 Finance Dashboard — static/finance/index.html

A **standalone HTML page** served at `/finance`. Matches the main app's Paper & Ink editorial design system exactly (same CSS tokens, fonts, light/dark mode).

Design system: - Fonts: `Instrument Serif` (headings/display values), `JetBrains Mono` (labels), `Inter` (body) - CSS custom properties: `--paper`, `--ink`, `--rule`, `--clay`, `--mansa` (#A68A3E light / #D4B86A dark) - Theme persistence: `localStorage('cassanoval-theme')`; `data-theme` attribute on `<html>`

Layout: 3-column grid — `sidebar (220px)` | `main content` | `chat panel (340px)`

6 navigation sections rendered by JS:

Section	Content
Overview	Net worth card, income/expense stats, cash flow bar chart (Chart.js 4.4), accounts preview
Rekening	Full accounts list grouped by type with asset/liability totals
Portfolio	Investment holdings table: qty × price, P&L, total market value
Budget Goals	Category goals with actual vs limit progress bars
Transaksi	Paginated recent transactions with account and category filters
Tagihan Rutin	Upcoming recurring bills sorted by next due date

Chat panel: Sends to `/api/chat` with `agent: 'budget'`. After each Mansa reply, `loadDashboard()` is called to refresh all displayed data.

Auto-refresh: `setInterval(() ⇒ loadDashboard(true), 5000)` polls the dashboard every 5 s (silent; a JSON signature diff skips re-render when nothing changed), so transactions added from anywhere appear within ~5 s. The **"Transaksi Terakhir"** card (`recent_transactions`, top 8) is ordered by `created_at` (date + time) server-side, so the most recently added transaction always shows first.

Chart.js theming: `getCSSVar()` reads current theme tokens at render time; `updateChartTheme()` called on theme toggle to update chart colors without recreating the canvas.

---

13. API Reference

POST /api/chat

```
Request:  { "message": "add task: buy milk", "agent": "task" }
Response: { "agent": "task", "response": "Done! Added 'buy milk' ... " }
```

GET /api/tasks

```
{ "tasks": [ ... ], "stats": { "total": 5, "pending": 3, "completed": 2, "high_priority": 1 } }
```

GET /api/notes

```
{ "notes": [ ... ], "total": 23 }
```

GET /api/budget/summary

Returns current balance, monthly totals, and recent transactions. Handles both old flat-list format and new dict format (backward-compatible). `recent_transactions` is sorted by `created_at` (date + time) descending, so the latest entry is first.

```
{ "balance": 4500000, "total_income": 5000000, "total_expense": 500000,
  "monthly_income": 5000000, "monthly_expense": 500000, "recent_transactions": [ ... ] }
```

GET /api/finance/dashboard

Rich data payload for the Finance Dashboard page at `/finance`. `recent_transactions` (top 50) is sorted by `created_at` (date + time) descending so the newest transaction is first.

```
{
  "net_worth": 18500000, "total_assets": 22000000, "total_liabilities": 3500000,
  "investment_value": 4200000,
  "accounts": [ ... ],
  "monthly_income": 8000000, "monthly_expense": 3200000, "current_month": "2026-05",
  "cash_flow": [ { "month": "2026-01", "income": 7000000, "expense": 2800000 }, ... ],
  "budget_goals": [ { "category": "Food", "monthly_limit": 1500000, "spent": 820000, "pct": 54 ]
  "investments": [ { "ticker": "BBCA", "quantity": 100, "buy_price": 8500, "current_price": 920(
                     "market_value": 920000, "pnl": 70000, "pnl_pct": 8.2 }, ... ],
  "recurring": [ { "description": "Spotify", "amount": 55000, "next_date": "2026-06-01", "days_u
  "recent_transactions": [ ... ],
  "net_worth_history": [ ... ]
}
```

GET /finance

Serves `static/finance/index.html` — the standalone Mansa Finance Dashboard page.

POST /api/crew/kickoff

Start a CrewAI pipeline or scraper job.

```
// Research pipeline
{ "topic": "CRISPR gene editing", "crew_type": "research" }

// DataAnalyst pipeline
{ "topic": "Analyze sales trends", "crew_type": "dataanalyst", "filename": "sales.csv" }

// Social Scraper
{ "topic": "AI trending", "crew_type": "scraper",
  "platforms": ["youtube", "tiktok", "reddit"],   // null = default 4 platforms
  "translate": false }

Response: { "job_id": "a1b2c3d4" }
```

GET /api/crew/status/{job_id}

Poll job status. Returns:

```
{ "status": "running" | "done" | "error",
  "result": "final output string (when done)",
  "outputs": {
    "task1_scout.txt": "...",
    "task6_final_report.md": "...",
    "youtube_summary.md": "...",
    "scraper_report.md": "..."
  },
  "logs": ["[Scraper] Starting...", "[YOUTUBE] Done — ..."],
  "error": "traceback (when error)" }
```

GET /api/crew/files

List CSV files available for DataAnalyst pipeline.

```
{ "files": [{ "name": "sales.csv", "size_kb": 42.3 }] }
```

POST /api/upload

Upload a CSV file for DataAnalyst pipeline. Form-data with `file` field.

POST /api/budget/scan-receipt

Upload a receipt image for automatic expense parsing. Form-data with `file` field.

---

14. Key Concepts Explained

Technology stack

Technology	Version	How loaded
React	18	CDN (`unpkg.com`)
ReactDOM	18	CDN
Babel Standalone	latest	CDN — compiles JSX in the browser
Press Start 2P	font	Google Fonts
VT323	font	Google Fonts

Why single-file React with Babel?

Normally React needs a build tool (Vite, webpack). Babel Standalone compiles JSX to plain JavaScript **inside the browser at runtime**. This adds ~1-2 seconds of load time but eliminates any build process.

Key Variables

```
// Auto-detects if opened as file:// and falls back to localhost
const API = (
  window.location.protocol === 'file:' ||
  window.location.origin === 'null'
) ? 'http://localhost:8000' : window.location.origin;
```

```
const P = { bg0:'#0a0a0f', grn:'#00ff41', ... }  // 8-bit color palette
const SP = { task: [[x,y,color], ...], ... }       // pixel sprite data
const AGENTS = { task: { name:'QUEST', ... }, ... } // agent config
```

Component Tree

```
App
├── Sidebar            ← left panel, agent selector
│   └── Avatar         ← pixel art SVG sprite
├── div (main chat)
│   ├── header bar     ← shows active agent name + dot indicators
│   ├── div (messages) ← scrollable message area
│   │   ├── Bubble[]   ← one per message (user or agent)
│   │   └── TypingIndicator ← animated dots while waiting
│   └── ChatInput      ← textarea + send button
└── DataPanel          ← right panel, contextual data
    ├── TaskWidget      ← stats + pending tasks list
    ├── NotesWidget     ← recent notes list
    ├── BudgetWidget    ← balance + recent transactions
    ├── NewsWidget      ← quick topic buttons
    ├── CodingWidget    ← quick curriculum buttons
    └── ScheduleWidget  ← quick action buttons
```

State Management

All state lives in the `App` component:

```
const [activeAgent, setActiveAgent]         // which tab is selected
const [messagesByAgent, setMessagesByAgent] // { task: [msg, ...], notes: [...], ... }
const [isLoading, setIsLoading]             // true while waiting for API response
const [panelData, setPanelData]             // { tasks:{}, notes:{}, budget:{} }
```

`messagesByAgent` is an object keyed by agent ID. Each agent has its own message array, so switching agents shows the conversation history for that agent — messages never mix.

Pixel Art Sprites

```
const Sprite = ({ pixels, size = 28 }) => (
  <svg viewBox="0 0 16 16" style={{ imageRendering:'pixelated' }}>
    {pixels.map(([x, y, color], i) => (
      <rect key={i} x={x} y={y} width={1} height={1} fill={color}/>
    ))}
  </svg>
);
```

Each sprite is a 16×16 grid. Pixels are stored as `[x, y, color]` tuples. The `imageRendering: pixelated` CSS property prevents the browser from anti-aliasing when the SVG is scaled up.

Connection Health Check

```
async function checkConnection() {
  try {
    const r = await fetch(`${API}/api/tasks`, { signal: AbortSignal.timeout(4000) });
    document.getElementById('conn-banner').style.display = r.ok ? 'none' : 'block';
  } catch {
    document.getElementById('conn-banner').style.display = 'block';
  }
}
checkConnection();           // run immediately on page load
setInterval(checkConnection, 15000); // recheck every 15 seconds
```

> Shows a red banner at the top if the backend is unreachable. Clears automatically when connection is restored.

sendMessage() — The Core Function

```
async function sendMessage(text) {
  // 1. Append user message to local state immediately (optimistic UI)
  setMessagesByAgent(prev => ({ ...prev, [activeAgent]: [...prev[activeAgent], userMsg] }));
  setIsLoading(true);

  // 2. POST to backend
  const res = await fetch(`${API}/api/chat`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ message: text, agent: activeAgent }),
  });

  // 3. Append agent response to state
  setMessagesByAgent(prev => ({ ...prev, [activeAgent]: [...prev[activeAgent], agentMsg] }));

  // 4. Refresh the data panel (tasks/notes/budget may have changed)
  await fetchPanelData(activeAgent);
}
```

---

14. Key Concepts Explained (continued)

What is a LangChain `@tool`?

```
@tool
def add_task(title: str, priority: str = "medium") -> str:
    """Add a new task. Args: title, priority ('high'/'medium'/'low')."""
    ...
```

The `@tool` decorator wraps a regular Python function so that: 1. The LLM can "see" it — the docstring becomes the tool's description 2. The LLM can "call" it — the type hints tell the LLM what arguments to provide 3. The agent framework can execute it — result is fed back to the LLM

The LLM never runs Python code. It generates a structured JSON object like:

```
{ "tool": "add_task", "args": { "title": "buy milk", "priority": "high" } }
```

LangGraph sees this, runs the actual `add_task()` function, and returns the result to the LLM.

What is a `CompiledStateGraph`?

LangChain 1.x replaced `AgentExecutor` with LangGraph. An agent is now a graph (state machine):

```
START
  │
  ▼
[call_model] ──── LLM decides: call a tool? ────→ [execute_tools] ─┐
     ▲                                                             │
     └─────────────────────────────────────────────────────────────┘
                      │
                      │ LLM decides: done
                      ▼
                    END
```

The loop runs until the LLM produces a message with no tool calls. You invoke it with:

```
result = agent.invoke({"messages": [HumanMessage("add task: buy milk")]})
answer = result["messages"][-1].content
```

What is Lazy Loading?

```
self._agents: dict = {}

def _load_agent(self, name):
    if name not in self._agents:
        # expensive operation — only do it once
        self._agents[name] = create_task_agent()
    return self._agents[name]
```

The agent object (including the LLM connection) is only created the first time that agent is needed. If you only ever talk to the task agent, the other 5 agents are never initialized. This saves startup time and memory.

What is Chat History?

```
history = [
    HumanMessage(content="add task: buy milk"),
    AIMessage(content="Done! Added 'buy milk' with medium priority."),
    HumanMessage(content="actually make it high priority"),
    AIMessage(content="Updated! 'buy milk' is now high priority."),
]
```

Every message is passed to the LLM on the next call. This is how the LLM "remembers" what was said earlier. Without history, every message would be a fresh conversation.

The `HumanMessage` / `AIMessage` classes are LangChain wrappers around the standard chat message format that all LLM APIs use.

---

15. Dependencies

```
# Core AI / LangChain
langchain           – Core framework, @tool decorator, create_agent
langchain-mistralai – ChatMistralAI (connects to Mistral API)
langchain-community – DuckDuckGoSearchRun, DuckDuckGoSearchAPIWrapper
langchain-core      – HumanMessage, AIMessage, PromptTemplate
langgraph           – CompiledStateGraph agent execution loop
mistralai           – Direct Mistral AI client (used by SummarizerAgent)

# Multi-agent pipelines
crewai              – Agent, Task, Crew, LLM classes (pipeline orchestration)
crewai-tools        – SerperDevTool, FileWriterTool

# Web scraping
scrapegraphai       – ScrapeGraphAI v2: SearchGraph + SmartScraperGraph + Mistral extraction
                      (requires: playwright install chromium after pip install)

# Search providers (priority: LinkUp > Serper > DuckDuckGo)
linkup-sdk          – LinkUp deep search API client
duckduckgo-search   – DuckDuckGo search (free fallback)

# Google Calendar integration
google-api-python-client – Google Calendar API client
google-auth-httplib2     – HTTP adapter for Google auth
google-auth-oauthlib     – OAuth 2.0 flow for Google APIs

# Web server
fastapi             – Web framework (routes, middleware, request models)
uvicorn[standard]   – ASGI server that runs FastAPI
python-multipart    – Required by FastAPI for file uploads

# Utilities
python-dotenv       – Reads .env file into os.environ
requests            – HTTP client (URL fetching)
yfinance            – Yahoo Finance data (stock pipeline)
pandas              – DataFrame operations (DataAnalyst pipeline)
plotly              – Chart generation (DataAnalyst pipeline)
chromadb            – Vector store (research tools, wiki)
scikit-learn        – ML utilities (DataAnalyst pipeline)
scipy               – Statistical analysis (DataAnalyst pipeline)
openpyxl            – Excel file reading (DataAnalyst pipeline)
```

16. Setup & Configuration

Environment Variables (.env)

```
MISTRAL_API_KEY=your_key_here          # Required — https://console.mistral.ai/
LINKUP_API_KEY=your_key_here           # Optional — deep search
SERPER_API_KEY=your_key_here           # Optional — search fallback
GEMMA4_API_KEY=your_key_here           # Optional — Google AI Studio key for Phase 2
GEMMA4_2_API_KEY=your_key_here         # Optional — can be same key as GEMMA4_API_KEY
```

Without Gemma keys, Phase 2 agents auto-fall back to Mistral.

Initial data files

```
echo [] > data/tasks.json
echo [] > data/notes.json
echo [] > data/budget.json
```

Google Calendar (one-time setup)

1. Go to https://console.cloud.google.com/
2. Create project → Enable **Google Calendar API**
3. Create **OAuth 2.0 Client ID** (type: Desktop App)
4. Download JSON → rename to `credentials.json` → place in `credentials/`
5. First time you use the Schedule agent, a browser opens for authorization
6. After authorization, `credentials/token.pickle` is saved automatically

Running the project

```
# Install dependencies
pip install -r requirements.txt

# Web mode (recommended)
$env:PYTHONUTF8=1; python server.py
cd "C:\Users\muham\OneDrive\Dokumen\Python\ai_python"
# Open: http://localhost:8000

# CLI mode (chat agents only)
$env:PYTHONUTF8=1; python main.py

# Run Ibn Al-Haytham pipeline directly
$env:PYTHONUTF8=1; python crewai_agents.py --topic "Your research topic"
```

Why `$env:PYTHONUTF8=1`?

Windows uses `cp1252` as the default terminal encoding. This forces Python to use UTF-8 everywhere — required for Indonesian characters, emoji, and any non-ASCII text in AI responses.

$env:PYTHONUTF8=1; python server.py *Documentation updated 2026-05-15 — CassanovaL Personal AI Multi-Agent Assistant Stack: Python 3.12 · LangChain 1.x · LangGraph · CrewAI · ScrapeGraphAI v2 · Mistral AI · Gemma 4 · FastAPI · React 18*
